

Design Practice: Functional Requirements

Project Idea

Describe your project idea. This could be a hardware project, a website, a mobile app, a desktop software, or an AR or VR app, basically anything that has a software component. Briefly describe your project idea.

This project is a gamified learning platform designed to make education more engaging, simultaneously helping users find career opportunities. It will cover a range of topics, including calculus, programming, and Boolean algebra, presenting each subject as an interactive game. The app will feature an actual 3D video game environment to keep users' attention and improve overall satisfaction. Users will be able to earn rewards, ranks, and industry-recognized certifications by completing a variety of projects and challenges.

The app supports three account types: students, professors, and employers, to facilitate learning, evaluation, and job placement. It also includes AI-powered tutoring to assist students with difficult concepts. The goal of the platform is to increase learning engagement and better connect educators, students, and employers.

Functions

List the functions that your software can be divided into. For each function, describe the inputs coming into it and the output returned by it.

User Account Management

Function: Will store user account information and will control how the user interacts with the app.

Input: User registration data including usernames, passwords, and email addresses.

Output: Access level based on the user's role (student, professor, employer). This determines

what the user can see and what is displayed on the dashboard.

Job matching system

Function: Will be an in-game hub where students can display progress, communicate with professors and match with employers.

Input: Students will enter profile data, that includes certificates, progress, completed projects. Employers will put in job postings and required qualifications. Professors will be able to provide recommendations.

Output: Potential job application links, and recommended employers based on users' profiles and job requirements.

Gamified learning system

Function: Will convert complex topics such as calculus, programming, and Boolean algebra into a 3D interactive learning environment. The app will present content in a modern open-world video game format featuring dungeons, quests, boss battles, and co-op. This will make learning more engaging and encourage players to collaborate.

Input: Challenge parameters such as quest type, boss difficulty, co-op requirements, and subject dungeon level. (E.g., Pre-calculus Level 1 dungeon, Calculus 1 Level 2 dungeon, Calculus 2 Level 3 dungeon).

Output: A final dungeon progress grade (A-C) based on weighted performance across all challenges within a dungeon. The boss battle will act as an optional bonus event that contributes up to 20% of the total progress grade. Scoring below 75% after completion of the dungeon will result in the player restarting. However, earning a passing grade allows the player to move on and earn a certificate that lets them enter the next dungeon.

AI tutoring system

Function: Will provide AI tutoring assistance through an in-game NPC (represented as monks) located across different towns. Students will be able to exchange in-game currency earned from completing challenges for lessons and hints to get past challenges inside dungeons.

Input: Student-selected subject or challenge within a dungeon, along with in-game currency used to access tutoring services.

Output: AI NPC provides players with lessons and hints to help them overcome challenges. These lessons and hints can be accessed at any time once purchased.

Reward and progression system

Function: Will be a logbook that tracks a player's progress throughout the game.

Input: Completed boss battles, challenges, dungeons, quests, time spent, and accuracy.

Tyler Velez

Output: Experience gained from completed challenges. Rewards such as in-game currency, badges, and certifications.

Pseudocode

Choose any ANY FOUR functions and write pseudo code for each of them. You should not limit yourself with what you know to be possible in Python. For example, your software might need to take input from hardware sensors and you haven't done that yet but you can assume and write your pseudocode with the understanding that it is possible. Use abstraction with sufficient details to help your grader to understand how your software works.

User Account Management

Get username and password from user

IF username already exists

 Return "Username already exists"

Else

 Create new account

 Return "Welcome to the app"

Get user role selection

IF user selects student

 Set access to student dashboard

Else if user selects professor

Set access to professor dashboard

Else

Set access to employer dashboard

Reward and progression system

For challenges completed in dungeons

Add experience based on difficulty

IF player challenge is easy

Add small experience

Give LV 1 badge

ELSE IF challenge is medium

Add medium experience

Give LV2 badge

ELSE IF challenge is hard

Add hard experience

Give LV3 badge

IF player beats dungeon boss

Add Bonus experience

Give Boss badge

IF player earns 75% or greater overall in dungeon

Give certificate

Gamified learning system

Set total score to 0

Set challenge counter to 0

FOR each challenge in dungeon

Increase challenge counter by 1

Get player answer

Check player's answer

Add score based on accuracy to total score

Run boss battle

Add boss battle score (worth 20%) to total score

Calculate total score

IF total score is 75% or greater

Give certificate

Unlock next dungeon

Else

Restart dungeon

AI tutoring system

FOR AI assistance in towns

Check player wallet

IF player wallet < 1000

Return "Player has insufficient funds"

ELSE

IF player wallet >= 5000

Give full explanation

ELSE IF player wallet >= 3000

Give four lessons

Give four hints

ELSE IF player wallet >= 1000

Give one lesson

Give one hint

Tyler Velez

[Optional for extra credit] Test tables for ANY FOUR functions

For each of the FOUR functions create test tables.

User Account Management

A = Student selected

B = Professor selected

C = Employer selected

D = Username exists

Q = $\sim D$ (Username available)

A	B	C	D	Q	Subexpression
0	0	0	0	1	$\sim A \sim B \sim C \sim D$
0	0	0	1	0	
0	0	1	0	1	$\sim A \sim B C \sim D$
0	0	1	1	0	
0	1	0	0	1	$\sim A B \sim C \sim D$
0	1	0	1	0	

0	1	1	0	1	$\sim A^{\wedge} B^{\wedge} C^{\wedge} \sim D$
0	1	1	1	0	
1	0	0	0	1	$A^{\wedge} \sim B^{\wedge} \sim C^{\wedge} \sim D$
1	0	0	1	0	
1	0	1	0	1	$A^{\wedge} \sim B^{\wedge} C^{\wedge} \sim D$
1	0	1	1	0	
1	1	0	0	1	$A^{\wedge} B^{\wedge} \sim C^{\wedge} \sim D$
1	1	0	1	0	
1	1	1	0	1	$A^{\wedge} B^{\wedge} C^{\wedge} \sim D$
1	1	1	1	0	

Gamified learning systemA = score $\geq$ 75%

B = boss passed

Q = If score is 75% $\geq$ unlock next dungeon

A	B	Q	R	Subexpression
0	0	0	0	
0	1	0	0	
1	0	1	1	$A^{\wedge} \sim B$
1	1	1	1	$A^{\wedge} B$

Reward and progression systemA = Score $\geq$ 75%

B = Boss passed

Q = Certificate earned

A	B	Q	Subexpression
0	0	0	
0	1	0	
1	0	1	$A \wedge \sim B$
1	1	1	$A \wedge B$

AI tutoring system

A = wallet < 1000

B = 1000 - 2999

C = 3000 - 4999

D = >= 5000

Q = Full explanation

A	B	C	D	Q	Subexpression
0	0	0	0	0	
0	0	0	1	1	$\sim A \wedge \sim B \wedge \sim C \wedge D$
0	0	1	0	0	
0	0	1	1	0	
0	1	0	0	0	
0	1	0	1	0	
0	1	1	0	0	
0	1	1	1	0	
1	0	0	0	0	
1	0	0	1	0	
1	0	1	0	0	

Tyler Velez

1	0	1	1	0	
1	1	0	0	0	
1	1	0	1	0	
1	1	1	0	0	
1	1	1	1	0	

Flowchart

Draw a flowchart showing a high-level working of your software. Your high level flowchart need not go into the detail of each function and each test case. The ones that you deem most important, should be represented with sufficient detail in your flowchart. For example, for a software to run a traffic lights 4-way intersection, the flowchart would include functions for turning the lights green, orange and red but need not show the looping of the timer tracking how long a specific light has been green, red or orange.

